Journal of Semiconductors
JOS iopscience.iop.org/jos
www.jos.ac.cn

**Robotic computing system and embodied AI evolution: an algorithm-hardware co-design perspective**

Longke Yan, Xin Zhao, Bohan Yang, Yongkun Wu, Guangnan Dai, Jiancong Li, Chi-Ying Tsui, Kwang-Ting Cheng, Yihan Zhang, and Fengbin Tu

Citation: L K Yan, X Zhao, B H Yang, Y K Wu, G N Dai, J C Li, C Y Tsui, K T Cheng, Y H Zhang, and F B Tu, Robotic computing system and embodied AI evolution: an algorithm-hardware co-design perspective[J]. *J. Semicond.*, 2025, 46(10), 101201.

View online: https://doi.org/10.1088/1674-4926/25020034

**Articles you may be interested in**

A survey of FPGA design for AI era

Journal of Semiconductors. 2020, 41(2), 021402 https://doi.org/10.1088/1674-4926/41/2/021402

Artificial sensory neurons and their applications

Journal of Semiconductors. 2025, 46(1), 011606 https://doi.org/10.1088/1674-4926/24080039

Flexible artificial vision computing system based on $FeO_x$ optomemristor for speech recognition

Journal of Semiconductors. 2025, 46(1), 012604 https://doi.org/10.1088/1674-4926/24080004

Recent progress on artificial intelligence-enhanced multimodal sensors integrated devices and systems

Journal of Semiconductors. 2025, 46(1), 011610 https://doi.org/10.1088/1674-4926/24090041

Rational design of multinary copper chalcogenide nanocrystals for photocatalytic hydrogen evolution

Journal of Semiconductors. 2020, 41(9), 091706 https://doi.org/10.1088/1674-4926/41/9/091706

Reconfigurable computing: a promising microchip architecture for artificial intelligence

Journal of Semiconductors. 2020, 41(2), 020301 https://doi.org/10.1088/1674-4926/41/2/020301

关注微信公众号，获得更多资讯信息

REVIEWS

*Journal of Semiconductors*, 2025, 46, 101201
DOI: 10.1088/1674-4926/25020034
CSTR: 32376.14.1674-4926.25020034

# Robotic computing system and embodied AI evolution: an algorithm-hardware co-design perspective

**Longke Yan[1], Xin Zhao[1], Bohan Yang[1], Yongkun Wu[1], Guangnan Dai[1], Jiancong Li[1], Chi-Ying Tsui[1, 2], Kwang-Ting Cheng[1, 2], Yihan Zhang[1], and Fengbin Tu[1, 2, †]**

[1]Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, China
[2]AI Chip Center for Emerging Smart Systems (ACCESS), Hong Kong, China

**Abstract:** Robotic computing systems play an important role in enabling intelligent robotic tasks through intelligent algorithms and supporting hardware. In recent years, the evolution of robotic algorithms indicates a roadmap from traditional robotics to hierarchical and end-to-end models. This algorithmic advancement poses a critical challenge in achieving balanced system-wide performance. Therefore, algorithm-hardware co-design has emerged as the primary methodology, which analyzes algorithm behaviors on hardware to identify common computational properties. These properties can motivate algorithm optimization to reduce computational complexity and hardware innovation from architecture to circuit for high performance and high energy efficiency. We then reviewed recent works on robotic and embodied AI algorithms and computing hardware to demonstrate this algorithm-hardware co-design methodology. In the end, we discuss future research opportunities by answering two questions: (1) how to adapt the computing platforms to the rapid evolution of embodied AI algorithms, and (2) how to transform the potential of emerging hardware innovations into end-to-end inference improvements.

**Key words:** robotic computing system; embodied AI; algorithm-hardware co-design; AI chip; large-scale AI models

**Citation:** L K Yan, X Zhao, B H Yang, Y K Wu, G N Dai, J C Li, C Y Tsui, K T Cheng, Y H Zhang, and F B Tu, Robotic computing system and embodied AI evolution: an algorithm-hardware co-design perspective[J]. *J. Semicond.*, 2025, 46(10), 101201. https://doi.org/10.1088/1674-4926/25020034

## 1. Introduction

Robotics has made remarkable strides over the past few decades, pushing the boundaries of what machines can accomplish autonomously or in collaboration with humans. Various robotic systems have been researched and developed in many domains. For example, in healthcare scenarios, surgical robots like the da Vinci Surgical System perform minimally invasive procedures with precision beyond human capability, improving patient outcomes and redefining standards of care[1]. In manufacturing sectors, collaborative robots, or "cobots", are now common in factories, working alongside human operators to enhance productivity, safety, and flexibility on the production floor[2].

A typical robotic system comprises a physical entity as its "body" and an intelligent agent as its "brain". The entity serves as the platform for interaction with the physical world and can manifest in various forms, such as manipulator arms, humanoid robots, quadrupeds, or drones, tailored to specific applications. The agent, conversely, comprises three fundamental subsystems: sensory, computing, and actuation. The sensory system employs various sensors (e.g., cameras, LiDAR) to gather data about the robot's internal state and the external environment. The actuation system, composed of servomotors, drives, and transmissions, enables the entity's movement. Bridging these is the computing system, which integrates intelligent robotic algorithms and their supporting hardware to intelligently connect sensory data with robotic actions. This computing system is pivotal, as it actualizes the desired robotic functionalities through sophisticated algorithms and dictates the system's performance in terms of accuracy and latency via its hardware foundation.

Advancements in robotic computing systems are increasingly characterized by algorithms delivering high intelligence and accuracy. For example, in perception, robotic computer vision techniques[3] enable robots to interpret complex camera data with minimal human oversight, a leap from traditional arms relying solely on internal sensors. In motion planning, the rapidly-exploring random tree (RRT)[4] and its variants have largely replaced classical methods like Dijkstra and A*[5] for high-degree-of-freedom (DOF) robots. Furthermore, for action tasks, model predictive control (MPC) is now widely adopted for tasks like quadrupedal locomotion, offering superior mobility on challenging terrains compared to traditional proportional-integral-derivative (PID) control. However, these traditional methods often rely on precise models of the robot and its environment, limiting their functionality in complex, human-like tasks and their generalizability to unknown scenarios.

In recent years, the confluence of robotics and AI has ushered in the era of embodied AI. This transition leverages data-driven AI models, enabling robots to process natural modalities (like language and complex visual scenes) and execute increasingly intelligent tasks. Unlike traditional robotics, which often requires extensive manual programming for specific tasks, embodied AI aims for greater dexterity and general-

Correspondence to: F B Tu, fengbintu@ust.hk
Received 26 FEBRUARY 2025; Revised 27 APRIL 2025.
©2025 Chinese Institute of Electronics. All rights, including for text and data mining, AI training, and similar technologies, are reserved.

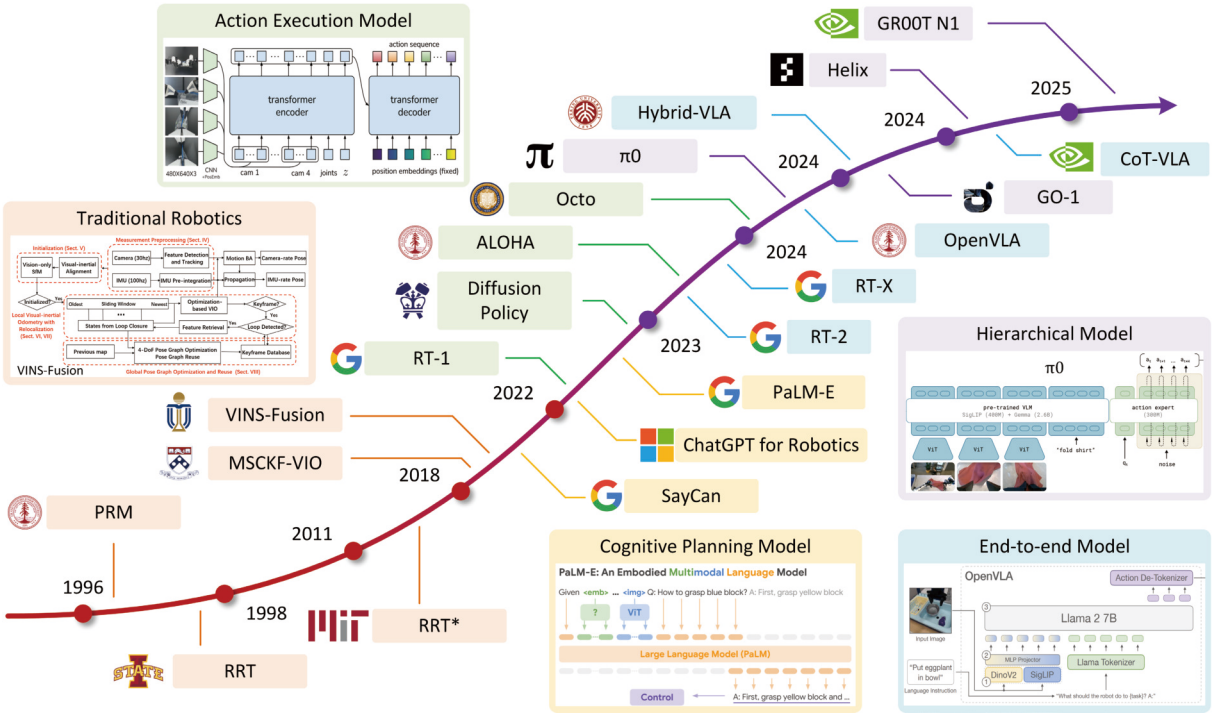



Fig. 1. (Color online) The evolution timeline of robotics and embodied AI. Traditional robotics has been developing since the last century, which is based on precise mathematical models of robots and environments. Because of the prosperity of AI, many action execution models have been proposed to transfer successful AI models into conducting more complex and dexterous robot actions since 2022. Meanwhile, since the emergence of LLMs, many cognitive planning models have also been proposed to leverage LLMs' ability to understand natural language and handle long-horizon planning. Therefore, combining an action execution model and a cognitive planning model into a hierarchical model has attracted much attention from academia and industry. Recently, end-to-end solutions based on VLA models are emerging as a new trend for embodied AI, due to the multi-modality processing capability in one system.

ization. Since 2022, many researchers have integrated successful AI models, such as Transformers and diffusion models, as **action execution models** to achieve more dexterous manipulations, like dual-arm coordination[6–8]. At the same time, the emergence of large language models (LLMs) like ChatGPT has paved the way for natural language understanding and long sequence processing on robots, so many **cognitive planning models** with the abilities of multimodal perception and long-horizon planning have been proposed since 2023[9, 10]. More recently, a trend has emerged integrating these two approaches into **hierarchical models**[11–14]. These typically employ a cognitive planning model at a higher level for understanding and reasoning, and an action execution model at a lower level for dexterous action control. Concurrently, researchers are exploring **end-to-end models**, often involving vision-language-action (VLA) architectures finetuned on large embodiment datasets to process multiple modalities in one system[15–18]. These end-to-end approaches represent an ambitious alternative competing with hierarchical models for state-of-the-art performance. The evolution timeline is summarized in Fig. 1.

Despite the rich functionalities and high accuracy offered by modern robotic and embodied AI algorithms, achieving an appropriate balance among system metrics such as accuracy, latency, and power consumption remains challenging, especially when relying on general-purpose computing hardware (e.g., CPUs, GPUs). While these platforms provide broad support for various workloads, they often fail to meet stringent real-time and energy-efficiency requirements. Consider RRT*[19], an advanced motion-planning algorithm. On CPUs, RRT* typically requires several seconds to converge to acceptable accuracy, rendering it impractical for dynamic scenarios. By contrast, GPUs offer significant speedups (milliseconds) but typically consume hundreds of watts, precluding their use in power-constrained systems like mobile robots or drones[20]. Moreover, standard hardware optimizations (e.g., caching, vectorization) may provide limited benefit due to the unique compute and memory access patterns prevalent in robotics algorithms[21]. This performance-efficiency gap is exacerbated by the rise of embodied AI, where large-scale models (often with billions of parameters) demand immense computational resources and data throughput, straining existing hardware, particularly at the edge.

Therefore, the development of specialized robotics-specific computing hardware is essential for enabling real-time, energy-efficient operation within constrained budgets. **Algorithm-hardware co-design** has emerged as the primary methodology for tackling this challenge, which analyzes algorithm computational behaviors on hardware and optimizes them through algorithm optimization and hardware innovation to gain substantial system-wide benefits, as outlined in Fig. 2. Specifically, traditional robotics and embodied AI algorithms' computational behaviors inherently exhibit common properties such as parallelism, locality, sparsity, and similarity, all of which create opportunities for algorithm-hardware co-design. On the algorithmic side, algorithm optimizations can be introduced to better leverage these inherent properties, reducing computing and storage complexity while exposing additional hardware support opportunities. For instance, Suleiman *et al.* exploited inherent data symmetric sparsity pat-

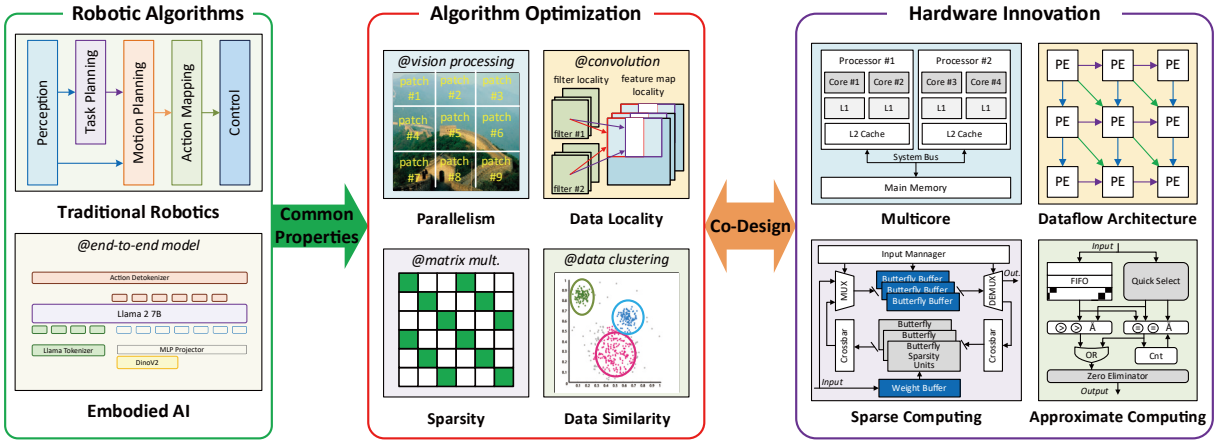



Fig. 2. (Color online) Robotic computing system and embodied AI evolution: an algorithm-hardware codesign perspective. Traditional robotics and embodied AI algorithms' computing behaviors inherently exhibit common properties. From the algorithm perspective, these properties are leveraged to optimize the computational behaviors of robotic algorithms and to reveal additional opportunities for hardware support. From the hardware perspective, innovations guided by these algorithmic properties address the balance of performance and energy efficiency across architectural and circuit levels.

terns to reduce memory requirement, thereby eliminating costly off-chip memory access through a simple memory wrapper and enabling the fully on-chip integration opportunity[22]. The sparsity patterns can also be found in Transformers, which was utilized by Wang through asymptotic sparsity speculation to predict sparsity patterns in varying attention output and skip their computation[23]. On the hardware side, these properties motivate hardware innovations from architecture to circuits. As an example, Li *et al*. leveraged inherent data parallelism by processing the stereo matching process along four paths in a deep pipeline at the architectural level, while developing an ultra-bandwidth SRAM that utilizes the unique memory access characteristic at the implementation level[24]. In a similar way, Tu *et al.* designed a Transformer accelerator to leverage the sparsity in its attention mechanism, which uses a sparse attention scheduler for efficient workload management at the architectural level and integrates transposition directly into the CIM macro to minimize data movement at the circuit level[25]. To sum up, by aligning the strengths of both domains, the algorithm-hardware co-design facilitates enhanced performance and efficiency of robotic computing systems.

The evolution of embodied AI and the requirement for the algorithm-hardware co-design are also increasingly demonstrated by industry prototypes. Systems like Tesla's Optimus[26], Agility Robotics' Digit[27], and Figure's Helix[13] showcase sophisticated capabilities in complex environments such as manufacturing floors and logistics warehouses, with ongoing real-world testing. Figure's Helix, for instance, employs a hierarchical approach: a 7B-parameter VLM acts as a high-level planner (System 2), while an 80M-parameter Transformer serves as a low-level controller (System 1), managing 35 DOFs. Trained on substantial teleoperation data (~500 h), Helix demonstrates impressive zero-shot generalization, leveraging the semantic understanding of VLMs and the sequential modeling power of Transformers. The Helix robot also serves as an illustrative example of algorithm-hardware codesign. A key challenge during Helix's deployment was the substantial frequency mismatch between the high-frequency action controller (System 1) and the lower-frequency planning model (System 2), leading to potential temporal misalignment during real-world operations. To mitigate this, the Helix development team implemented a hardware-aware training methodology. This involved introducing a deliberate temporal offset between the inputs of the two systems during the training phase, effectively simulating the asynchronous execution characteristic of the target hardware deployment. This co-design approach ensures that the low-level controller (System 1) can robustly integrate the latest proprioceptive and sensory feedback with the most recently available latent command representations from the high-level VLM (System 2). However, this highlights ongoing opportunities for further optimization through dedicated hardware accelerators developed via co-design principles.

The organization of this survey is summarized as follows. Section 2 discusses traditional robotics algorithms and corresponding specialized hardware. We provide a detailed analysis of algorithms and hardware for perception, planning, and action, respectively. In Section 3, embodied AI algorithms alongside the three-step roadmap are presented as well as computing hardware acceleration works for basic embodied AI algorithms. In Section 4, we analyze the main challenges for embodied AI systems driven by both application and hardware advances and propose a future perspective of the three-layer technology stack and the opportunities in it. Finally, we conclude the survey in Section 4.

## 2. Traditional robotics algorithm and hardware

In this Section, we introduce the key algorithms of traditional robotics and their corresponding hardware acceleration techniques. Traditional robotic algorithms generally conduct five basic functional tasks, i.e., perception, task planning, motion planning, action mapping, and control, as shown in Fig. 3. Perception algorithms process sensory data and extract high-level information such as objects, positions, and maps. Based on the information, a robot decides what actions to take (task planning) and how to execute them (motion planning) in the future. Then a robot uses action mapping algorithms formulated by robot mechanisms to trans-

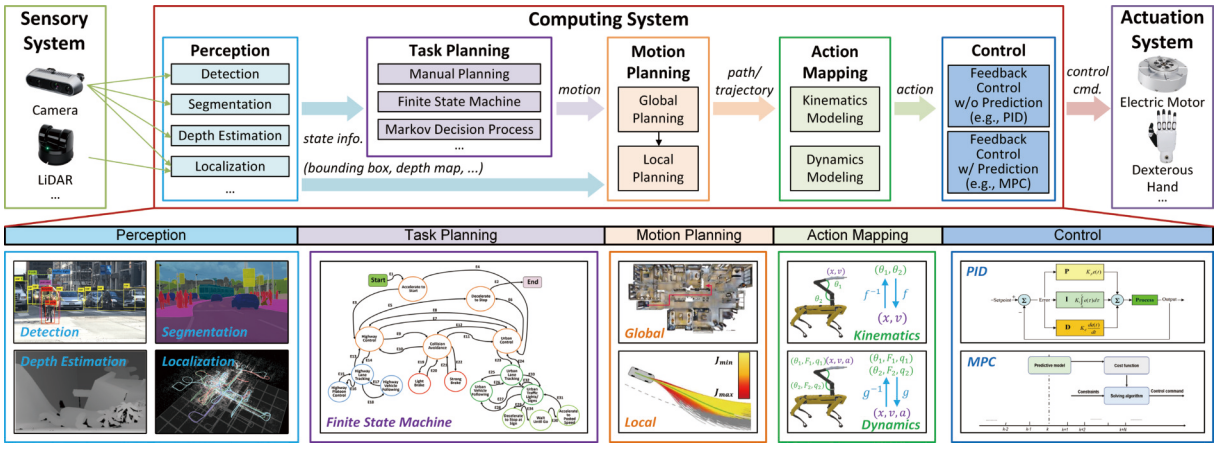



Fig. 3. (Color online) Overview of a typical computing system in traditional robotics. It is responsible for five basic tasks: perception, task planning, motion planning, action mapping, and control.

late paths/trajectories from motion planning into robot actions. Finally, control algorithms send command sequences or signals to the actuators to follow the motion of mechanical components based on reference actions in a feedback way. Each stage in this pipeline demands specialized algorithms and computing hardware that can deliver high accuracy, real-time performance, and high energy efficiency. These requirements are further enhanced in embedded and mobile systems, where limited resources and strict power budgets present significant design and implementation challenges.

## 2.1. Perception

Perception involves processing sensory data obtained from both internal and external sources—such as cameras and LiDARs—to construct a reliable and detailed understanding of the environment and the robot's own state. Without loss of generality, we introduce four typical perception tasks: detection, segmentation, depth estimation, and localization.

### 2.1.1. Detection

Detection is a fundamental vision task that can recognize the category and position of target objects in images or videos. It can be divided into the two-stage and single-stage methods. The two-stage methods, represented by RCNN series leveraging convolutional neural networks (CNNs)[28], first generate region proposals to obtain regions that may contain the target object, and then classify the region proposals and perform bounding box regression to get the category and position of the target. While accurate, the two-stage methods suffer from high computational complexity, which hinders inference speed. To address this, the single-stage methods like the YOLO series[29] skip the generation of region proposals and directly predict the category and bounding box position of the target from the input images, offering faster inference but often sacrificing some accuracy.

Various accelerators have been proposed to achieve real-time performance with low power. Lefebvre *et al.* proposed a convolutional imaging system-on-chip (SoC) that accelerates feature extraction and object detection by performing multiply-accumulate (MAC) operations directly within the sensor domain[30]. Gong *et al.* proposed an accelerator optimized for multi-scale semantic feature extraction (MSFE)-based detection frameworks[31]. It reduces computational overhead through parallel frame-difference calculation and leverages unified data compression and computation to address data redundancy in activation functions, object regions, and interpolation processes.

### 2.1.2. Segmentation

Segmentation aims to group pixels of the given image or video into a set of semantic regions, providing finer-grained visual information about object semantic labels, shapes, and positions. It can be divided into semantic segmentation, which labels regions with a single class, and instance segmentation, which distinguishes multiple objects of the same class (e.g., labeling different robots in an image with unique colors). The output of segmentation is typically a map of the same size as the input image, so traditional CNNs are not suitable as their feature map size decreases with each network layer. UNet is the most widely used segmentation network, which first introduces the upsampling operator to restore the segmentation map[32]. Another widely used approach is the DeepLab series, developed by Google, which employs dilated convolutions to capture multi-scale contextual information[33].

Segmentation is more data-intensive than detection as it requires pixel-level computations for every image or frame, so many works focus on designing accelerators for the segmentation algorithms. Vohra *et al.* proposed a low-energy imaging device with event detection and object segmentation[34]. It adopts analog background subtraction for event detection, preserving data locality. In addition, it restricts the subsequent neural network processing to specific regions of interest rather than the entire frame, avoiding the processing of other regions. To leverage the high energy efficiency of compute-in-memory (CIM), Guo *et al.* proposed a floating-point CIM-based architecture, which reduces internal bandwidth and area through high-bit full-precision and low-bit approximate-calculation multipliers with almost no decrease in accuracy[35].

### 2.1.3. Depth estimation

Depth estimation is extracting environmental depth information by processing images from two cameras from two points of view in the same scenario. The spatial distance information is estimated based on the disparities between the corresponding pixels searched by stereo matching algorithms and then is represented as disparity maps. There are two types of stereo matching algorithms, local methods and

global methods. Local methods calculate disparity based on the information of local adjacent pixels while trading accuracy for speed[36]. On the contrary, global methods extract disparity values from the entire image, which is more accurate but more time-consuming[37].

As stereo matching is the bottleneck of stereo vision, special hardware acceleration is needed to improve overall performance. Local matching is easy to accelerate for real-time performance as it. Sekhar *et al.* implemented the SAD algorithm on an FPGA and developed an efficient line buffering scheme for data parallelism and locality[38]. To deliver high-resolution and accurate disparity maps, Li *et al.* designed a high-performance and energy-efficient processor for a semi-global matching (SGM) algorithm by a high-throughput pipelined architecture with a dependency-resolving scheme to expose data parallelism, and further developed a customized ultra-high bandwidth SRAM by identifying unique data behavior to achieve highly energy-efficient memory access[24].

#### 2.1.4. Localization

Localization estimates the position and orientation, or the 6 DOF pose, of a robot in a given environment by calculating data from sensors such as cameras and LiDARs. There are two typical methods, simultaneous localization and mapping (SLAM) and visual-inertial odometry (VIO). SLAM is usually used to construct a map of indoor environments while localizing a robot in the map. It is usually formulated as a constrained nonlinear optimization problem and is more robust in long-term localization. It typically consists of four parts: front-end, back-end, loop detection, and mapping. On the contrary, VIO methods and other odometry methods[39], usually used in outdoor environments, utilize Kalman-filtering methods to calculate relative poses from a starting point, but they don't need to construct an explicit map.

As mobile robots need to respond to their surroundings in real time under the constraint of a limited power budget, many localization accelerators were proposed. Liu *et al.* designed an energy-efficient FPGA accelerator for the back-end optimization in SLAM to fully utilize data sparsity, locality, similarity, and pipeline opportunities[40]. Suleiman *et al.* implemented a fully integrated VIO accelerator in a single chip to eliminate massive energy-consuming data transfer[22]. It further exploits compression and sparsity methods to reduce storage requirements while leveraging rescheduling and parallelism for high-throughput processing. To adapt to different operating scenarios, Gan *et al.* presented a unified localization algorithm framework by identifying common kernels in SLAM, registration, and VIO[41]. Based on the insight about latency in the front end and latency variation in the back end, it co-designed an efficient front-end accelerator with task parallel processing and data locality memory scheme and an accelerator with a workload scheduler, which offloads the back end only when the latency could be reduced. To release the tedious efforts of manual accelerator designing and adaptation, Liu *et al.* proposed the Archytas framework to automatically generate synthesizable accelerators from algorithm descriptions to meet power, latency, and resource specifications[42]. This is achieved by constructing a Macro Data-Flow Graph (M-DFG), optimizing the hardware template, and solving a constrained optimization problem for hardware generation.

### 2.2. Task planning

Task planning aims to determine a sequence of future motions of a robot for specific tasks based on environmental information from perception. Initially, the motions to complete a given task are typically decomposed and scheduled by human experts and engineers. As the motions and environmental states within a task can be modeled as finite sets, a natural method to improve the planning intelligence is using finite state machines (FSMs) to schedule possible motion states with transitions from the perception information[43]. For example, in autonomous driving, the winning team of the DARPA Urban Challenge employed hierarchical FSMs to implement the behavior generation component[44]. In this approach, the driving task is broken down into high-level behaviors (states) such as "drive down road", "handle intersection", and "achieve zone pose", each of which is further divided into simpler sub-behaviors aimed at accomplishing the autonomous driving planning task. Despite the effectiveness of FSMs in structured environments, they struggle to account for uncertainty and dynamic changes in the environment. Therefore, many works have sought alternative frameworks that can better handle such uncertainties. One promising approach involves formulating task planning as a partially observable Markov decision process (POMDP), allowing for more sophisticated decision-making in uncertain contexts. For example, Liu *et al.* proposed an intention prediction method for autonomous vehicles, using a hidden Markov model (HMM) to forecast the future driving intentions of human-driven vehicles in a mixed-traffic environment[45].

The task planning algorithms are typically executed on microcontroller units (MCUs) to provide real-time performance with limited hardware resources. However, as the intelligence of task planning increases, there is a growing demand for higher computing power to process massive data and support intelligent algorithms, which MCUs are increasingly unable to fulfill. Therefore, many works now use systems-on-chip (SoCs) as the primary computing platforms. A notable example is Tesla's full self-driving (FSD) computer, the world's first system designed to handle the resource-intensive tasks required for autonomous driving[46]. This SoC integrates industry-standard components, such as CPUs, ISPs, and GPUs, alongside custom neural network accelerators, achieving impressive performance at a low power consumption.

### 2.3. Motion planning

Motion planning generates collision-free trajectories to execute motions determined from task planning. It involves two key categories: global planning and local planning. Global planning focuses on finding the shortest collision-free path from a starting pose to a target pose, while local planning refines trajectories based on constraints such as safety and physical constraints.

#### 2.3.1. Global planning

Global planning can be divided into two classes: the graph-search-based methods and the sampling-based methods. The graph-search-based methods abstract robot states or poses as nodes and robot motions as edges. The optimal path between two positions is searched serially from a start node to a target node by traversing a state space[47]. A* algorithm[5] is a heuristic extension to the well-known Dijkstra[48],

which applies heuristic knowledge to guide the search directions, reducing the iterations and overall operating time. However, graph-search-based methods are typically time-consuming for high-dimensional state space such as a robot with 6 DOF[49], so the sampling-based methods are proposed to find trajectories by probabilistic sampling with faster convergence. There are two commonly used algorithms: the probabilistic roadmap method (PRM)[50] and the rapidly-exploring random tree (RRT)[4]. PRM leverages sampling to accelerate graph-search-based methods, while RRT completes motion planning tasks in a single process. As RRT and its variants, such as RRT*[19], show outstanding efficiency in large-scale environments, they have been widely applied for commercial and industrial products.

Global planning can be compute- and memory-intensive within vast areas or high-dimensional spaces. To achieve real-time performance for the graph-search-based methods, Bakhshalipou *et al.* observed similarity patterns in the serial state searching in A* and accordingly developed a run-ahead state searching extension to the proposed accelerator named RACOD to predict future poses and perform collision checks[51]. For the sampling-based methods, Lian *et al.* optimized PRM with an update phase to prune the redundant edges and developed a scalable PRM accelerator named Dadu by adopting a hardware-friendly graph representation to expose spatial structures effectively[52]. Chung *et al.* optimized RRT with various parallel schemes to reduce computing complexity and memory requirements[53]. On the hardware side, they designed a prune-and-reuse strategy for fast response to dynamic environments. They also implemented an accelerator to process these parallel schemes and reduced the hardware complexity of PEs with approximate computing.

#### 2.3.2. Local planning

Local planning is typically formulated as an optimization problem to compute (near-)optimal trajectories that satisfy diverse constraints. Based on the optimality criterion, there are three most important criteria: execution time, energy, and jerk[54]. Furthermore, some hybrid optimality criteria were also developed to achieve balanced performance. Many numerical methods are proposed to solve these optimization problems[55]. As the problem scales and the task intelligence increase, distributed optimization techniques such as the alternating direction method of multipliers (ADMM)[56] and data-driven methods are attracting more focus[57].

As solving optimization problems includes massive matrix operations such as rotation matrix transpose and matrix decomposition, it is a direct result that many GPU-based optimization solvers are developed. Chretien *et al.* developed a GPU implementation for motion planning formulated as a semi-infinite optimization program, which leverages data-independent and data-dependent parallelism to compute constraints and their gradients efficiently[58]. By using the abstraction of a factor graph, Hao *et al.* revealed the opportunities to reduce the scale of optimization problems and solve them in an incremental way, as well as the inherent sparsity in motion planning[59]. Based on the insight, they developed a hardware accelerator to leverage potential parallelism and sparsity and a framework to search for the optimal order to solve different optimization problems. Furthermore, Hao *et al.* proposed a hardware generation framework based on the concept of a factor graph to generate customized accelerators automatically for solving optimization problems in robotics[60].

### 2.4. Action mapping

Action mapping is translating mathematic paths/trajectories into physical robot actions and vice versa by modeling robot mechanisms. There are two types of robot mechanism models: kinematics and dynamics. Kinematics describes the motion of bodies without regard to the forces/torques that cause the motion. On the other hand, dynamics refers to the relationship between the external forces/torques on robotic objects and the motions they produce.

#### 2.4.1. Kinematics

Kinematics formulates the end-effector poses as a function of the joint variables. There are two kinematics processes: forward kinematics (FK) and inverse kinematics (IK). FK refers to calculating the end-effector poses from the joint variables through kinematics equations, while IK is an opposite and more complex process to calculate the joint variables from the target end-effector poses. There are two main types of algorithms for solving the IK problem[61]. The analytic methods use straightforward and closed-form solutions from algebraic and geometric analysis, which is reliable and precise with low DOF[62]. On the contrary, the numerical methods formulate IK as an optimization problem and they can be divided into three categories: Jacobian inverse methods, Newton methods, and Heuristic inverse methods.

The acceleration on IK is necessary as it is usually applied in real-time applications but is rather time-consuming, especially in high DOF robots. Tkachenko *et al.* developed an FPGA implementation for a geometrical IK method with hardware optimization such as data type reduction and module sharing[62], but it only works for specific robot mechanisms. To solve general inverse kinematics, Lian *et al.* designed a speculation-based Jacobian transpose algorithm to reduce iteration counts and implemented an accelerator to exploit the algorithmic parallelism in each iteration for real-time performance with high energy efficiency in high DOF applications[63].

#### 2.4.2. Dynamics

Dynamics provide the relationships between actuation and contact forces acting on robot mechanisms, and the acceleration and motion trajectories that result. Inverse and forward dynamics with their gradients are key kernels in many motion planning and control techniques. They can account for 30% to 90% of the total computational time of a typical nonlinear MPC algorithm[64]. To improve their performance, one of the state-of-the-art algorithms uses templating and code generation by leveraging robot morphology features into the code[65].

To meet the real-time requirement, many hardware accelerators are developed. Neuman *et al.* proposed a design methodology similar to Ref. [65], using robot topology and structure to exploit parallelism and matrix sparsity patterns in accelerator hardware, and implemented a parameterized accelerator for the gradients of dynamics[66]. Neuman *et al.* further leveraged robot topology to design scalable and flexible accelerators, focusing on topology traversals and large topology-

based matrices[67]. These patterns of parallelism and locality enable hardware optimization tailored to specific robot architectures. Yang *et al.* provided a multifunctional framework capable of handling various dynamics tasks without hardware reconfiguration by identifying inherent relationships, and developed two specialized pipelines to optimize data locality utilization and adapt to robot-specific sparsity[68].

### 2.5. Control

Control algorithms typically ensure that a robot and its components follow the reference action by the computation of direct commands like steering angles, throttle opening, and braking pedal in a feedback framework. There are two types of control algorithms: feedback control without prediction and feedback control with prediction[69]. PID control is one of the most popular algorithms of feedback control without prediction for commercial and industrial purposes due to its simplicity and efficiency. Another widely applied control algorithm is MPC, a prominent approach for feedback control with prediction, which can predict a robot's future behavior using models like dynamics and optimize a sequence of control commands to achieve better overall performance in dynamic environments[69].

When there is only one control task, it is easy to design a controller with PID or MPC. However, for robots with redundant DOFs, there are often multiple control tasks such as stability and safety maintenance, especially in quadruped robots and humanoid robots. An important approach in multi-task control is whole-body control, which assigns priorities to multiple tasks reasonably when they cannot be executed simultaneously. In terms of algorithm formulation, there are 2 types of whole-body control: closed-form methods and optimization methods[70]. In the first case, the final actions are derived after a sequence of algebraic operations based on task priorities (e.g., pseudo-inverse matrices, null-space projection)[71, 72]. Its key advantage is computational speed, making them particularly well-suited for real-time applications. In the second case, the problem is defined as an optimization problem (e.g., quadratic programming, convex optimization), and the task priorities are implicitly represented between the objective and the constraints[73, 74]. Though they demand greater computational resources, optimization-based WBC excels in scenarios requiring sophisticated task prioritization and contact force optimization, such as a smooth transition between dynamic gaits for quadrupedal robots[73].

For control algorithms that usually operate in high control rate scenarios (>1 kHz), it is demanding for CPUs and GPUs to achieve real-time performance with a limited power budget, so hardware acceleration is a necessary choice. There are numerous hardware acceleration works for PID control[75]. The high complexity of MPC often hinders their applications in high-speed and high-precision scenarios, so various hardware accelerators of MPC were proposed. For gradient-based optimization acceleration, Li *et al.* accelerated the quadratic programming solver of MPC on an FPGA[76], while Lin *et al.* proposed the pruning strategy to leverage parallel and sparse natures of the sampling-based MPC and physical model transformation scheme to reduce physical model computing complexity, and implemented a high-performance ASIC based on these methods[77].

## 3. Embodied AI algorithm and hardware

Embodied AI represents a significant evolution from model-driven traditional robotics, distinguished by its ability to perform more intelligent tasks by using data-driven methods. This transformation is motivated by two breakthroughs in the field of AI. The first advancement is based on the emergence of Transformers and diffusion models which have shown superior ability to handle long sequence and high dimensional data. These properties can enable robots to conduct more complex and dexterous tasks that traditional robotics is difficult to model, so many action execution models have been proposed based on Transformers and diffusion models[7, 8]. The second advancement arises from the emergence of LLMs and VLMs, which facilitate natural language understanding and long-horizon planning on robots, pushing the development of cognitive planning models[10, 78]. Based on the two trends, many hierarchical models from academia and industry such as Helix[13] and GR00T N1[14] have appeared to use a cognitive planning model as a perception and planning generalist and an action execution model as an action specialist. At the same time, the solution of an end-to-end model is still attractive in the embodied AI community and competing the state-of-the-art with other models, such as OpenVLA series[17, 79]. Furthermore, an end-to-end model is likely the final model architecture for general-purpose robots. Therefore, the evolution of embodied AI suggests a three-step roadmap from traditional robotics to a hierarchical model and ultimately to an end-to-end model, as shown in Fig. 4.

In this Section, we will discuss embodied AI algorithm advancements in the generations of the hierarchical model and the end-to-end model and then analyze the computing hardware developments for corresponding basic embodied AI algorithms.

### 3.1. Hierarchical model

#### 3.1.1. Cognitive planning model

A cognitive planning model is essential in a hierarchical model, enabling robots to understand human instructions and perform long-horizon planning based on the instructions and environmental information. These models typically leverage the advanced capabilities of LLMs and VLMs in handling extended sequences and extensive knowledge bases, thereby allowing sophisticated task execution that requires long-horizon planning.

A notable method is using LLMs as a foundation model to exploit their capability for long-horizon task decomposition and scheduling based on textual descriptions of environments, robots, and tasks. A representative example of this methodology is ChatGPT for Robotics[9], developed by Microsoft. This model integrates OpenAI's ChatGPT with a high-level function library that can be linked in the back end to the actual APIs in robots, allowing it to translate diverse robotic tasks from natural language instructions into a logical chain of high-level function calls without requiring platform-specific knowledge. It also develops a robust pipeline that supports zero-shot task planning and seamlessly adapts to various platforms. However, ChatGPT for Robotics relies on human feedback to ensure safety and improve quality, which limits its autonomous capabilities across different scenarios.

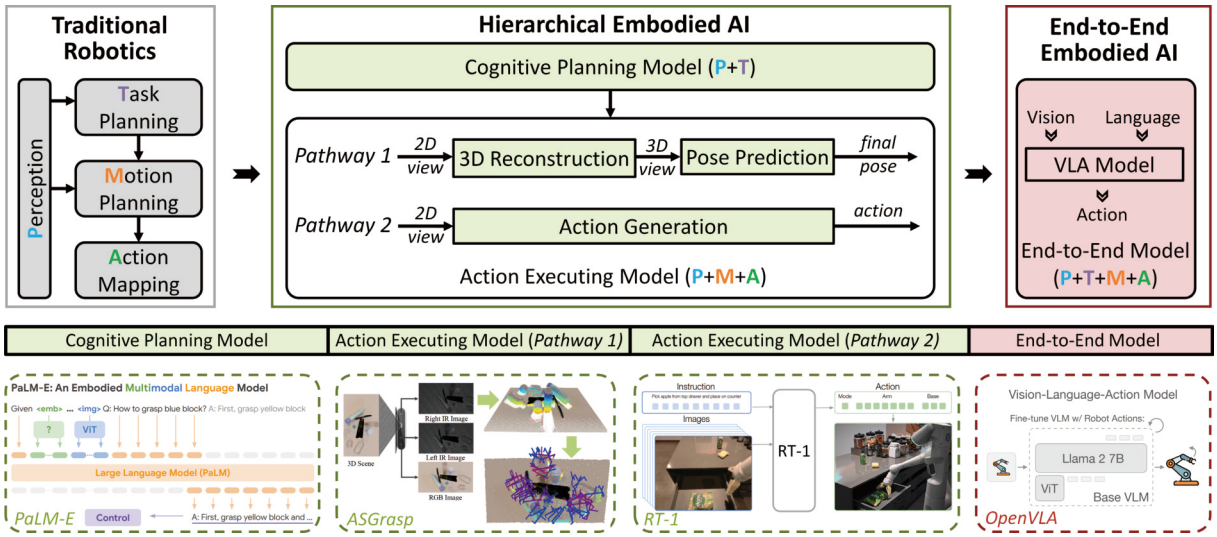



Fig. 4. (Color online) A perspective of the three-step embodied AI roadmap. The roadmap begins from traditional robotics with separate algorithms based on application-specific robot and task models. In the hierarchical model stage, cognitive planning models leverage natural language understanding and long-horizon reasoning of LLMs and VLMs for multimodal perception and general planning, while action execution models specialize in performing actions for specific tasks. Finally, the end-to-end model stage creates a fully integrated embodied AI model, enabling highly generalized and intelligent task execution across dynamic and unfamiliar environments.

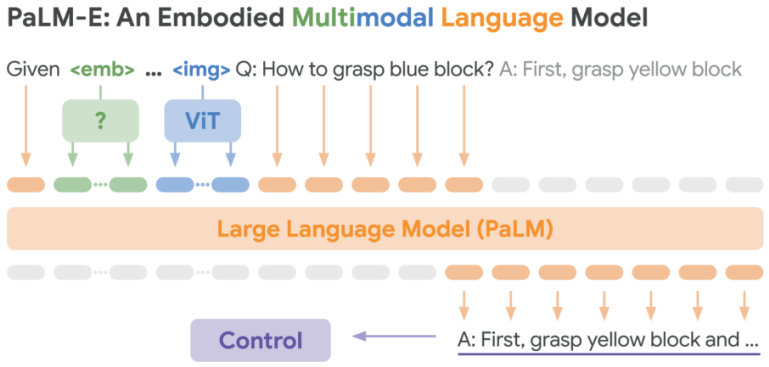



Fig. 5. (Color online) PaLM-E model architecture overview (from PaIM-E[10]).

To address this limitation, many works have focused on integrating autonomous feedback mechanisms to enhance planning quality and success rates across different robots and scenarios. For example, Inner Monologue[80] combines LLM-based planning with real-time environmental feedback. By utilizing a variety of perception models, such as CNNs and Transformers, for tasks including success detection, scene description, and human interaction, it enables dynamic and closed-loop reasoning. This approach also allows robots to adapt and recover from failures during long-horizon tasks, thereby improving the efficiency of high-level task planning, even when facing previously unseen tasks.

Despite these advancements, LLM-based cognitive planning models are limited by their reliance on textual inputs, which can result in the loss of critical information from other sensory modalities, such as visual data from cameras. To address this, there is a growing focus on developing multimodal large models like VLMs that can extract diverse semantic features from various sensors and reason about general robotic tasks. A representative example of this approach is PaLM-E[10], developed by Google. PaLM-E is a multimodal large model that combines VLM pretraining with fine-tuning on robotic data. It is built upon the 540-billion-parameter pretrained LLM backbone of PaLM[81] and integrates a 22-billion-parameter ViT[82] to process image information, along with several MLPs to map state estimation vectors and scene representations, as shown in Fig. 5. To create a unified embodied model, PaLM-E is trained end-to-end on multiple robotic tasks, enabling it to perform a wide range of embodied reasoning tasks, including robotic manipulation, visual question answering, and long-horizon planning. Additionally, PaLM-E demonstrates the ability to generalize across multiple robot configurations, enhancing its applicability in diverse operational contexts. While PaLM-E excels in controlled environments, additional refinements are necessary for its effective deployment in highly dynamic and complex real-world scenarios, and its large model size presents computational challenges. Overall, PaLM-E's capability to integrate multimodal inputs into a unified model represents a substantial progression in embodied AI, particularly in the development of multimodal large models for real-world robotics applications.

### 3.1.2. Action execution model

An action execution model is responsible for executing low-level actions scheduled by cognitive planning models. The goal of these models is to develop a specialized model based on task-specific embodiment data, enabling the successful completion of multiple actions while ensuring faster inference speeds compared to cognitive planning models. As

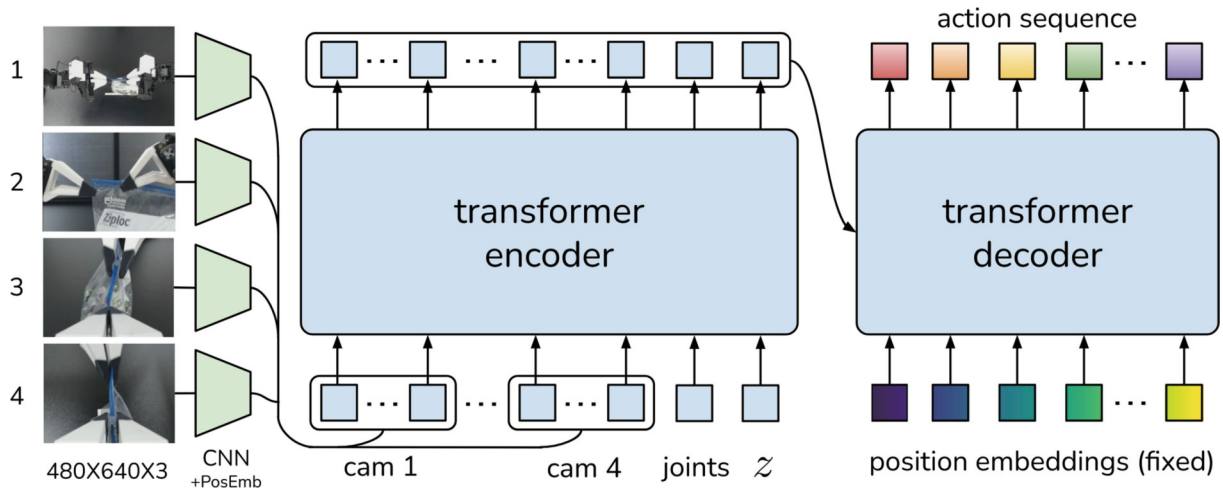



Fig. 6. (Color online) Model architecture of Action Chunking with Transformers (ACT) (from ALOHA[7]).

shown in Fig. 4, There are typically two types of pathways: pose prediction and action generation. The first pathway considers the determination of the final pose as the most important and challenging step during the whole operation, especially for manipulation. To fully perceive the objects' properties such as distance and deformability, 3D reconstruction is usually integrated into the pose prediction frameworks. In the second pathway, the solution is more aggressive to use an action generation model to control robot actions during the entire process and directly use visual images as the feedback to enable dexterous and agile actions.

**(1) Pose prediction.** The target of pose prediction is to find the optimal final pose for specific tasks. The process begins with constructing 3D geometric views through 3D reconstruction methods, which can provide rich information about target objects, including their position and shape. Based on 3D information, a pose prediction model outputs the optimal pose on the target objects. Subsequently, traditional motion planning algorithms generate a trajectory from the initial pose to the final pose, which is then translated into robot actions using action mapping algorithms. Recent works in pose prediction have primarily focused on robot manipulation to improve precision and reliability in various environments.

3D reconstruction is to generate three-dimensional object or scene representations using 2D images or other sensor data[83] Making it provides precise spatial models of objects and scenes to facilitate spatial and semantic information reasoning, especially in conducting fine-grained tasks such as manipulation. It can be an essential component in specific cases such as transparent or specular objects, while 2D images and point clouds exhibit limitations in these scenarios. For instance, methods relying solely on 2D images, such as grasp pose prediction, are restricted to 3-DoF due to the absence of 3D geometric information[84, 85], and point clouds often yield inaccurate or incomplete depth information when processing transparent or specular objects[86]. Current reconstruction methods can be typically categorized into three types: point cloud-based reconstruction, neural radiation field-based (NeRF) reconstruction, and 3D Gaussian Splatting-based (3D GS) reconstruction. Point cloud-based 3D vision models have seen increased application in robotics following the introduction of PointNet[87] by Stanford University. For example, ASGrasp[86], developed by the Samsung R\&D Institute, utilizes a multi-level convolutional gated recurrent unit (GRU) network to simultaneously capture visible point clouds of transparent objects and invisible point clouds of occluded surfaces. NeRF-based reconstruction leverages MLPs to provide a more comprehensive representation of 3D information compared to point clouds, thereby improving the success rate of robotic actions. A notable example is a mesh generator developed by Zhou *et al.*, which employs NeRF to learn the neural signed distance field (SDF) centered around an object, thereby obtaining its detailed 3D view[88]. Similarly, GraspNeRF[84] proposed by Dai *et al.* processes sparse multi-view RGB images to reduce inference time. It aggregates feature volumes through feature aggregation MLPs and constructs a complete 3D geometric scene representation via SDF. To address potential issues with unclear segmentation boundaries in NeRF, GaussianGrassper[89] developed by Zheng *et al.* employs 3D GS for scene reconstruction. This method involves generating a 3D Gaussian field through an MLP-based feature extraction network and performing geometric reconstruction via projection and rasterization operations. Comparative visualizations demonstrate that GaussianGrassper achieves higher reconstruction quality compared to NeRF-based methods such as GNFactor[90]. Furthermore, for dynamic scene reconstruction, AgiBot adopted a 4D GS reconstruction method to generate GS-rendered images, enhancing the model's ability to handle time-varying environments[91].

Pose prediction models play a crucial role in robotic grasping by processing 3D geometric data to determine optimal grasping poses. These models leverage advanced neural network architectures to enhance the accuracy and efficiency of robotic manipulation tasks. In ASGrasp[86], the grasping detection module utilizes GSNet to identify the final grasping pose. GSNet[92] operates by first generating feature vectors through a point encoder-decoder architecture based on ResUNet14. Subsequently, it determines the grasping pose through multiple cascaded MLPs, achieving an action frequency of approximately 10 frames per second (fps). Building on the high success rate and action frequency of GSNet, AnyGrasp[6] developed by Fang *et al.* also integrates GSNet as its geometry processing module. AnyGrasp further refines the optimal grasping pose using a temporal association module composed of

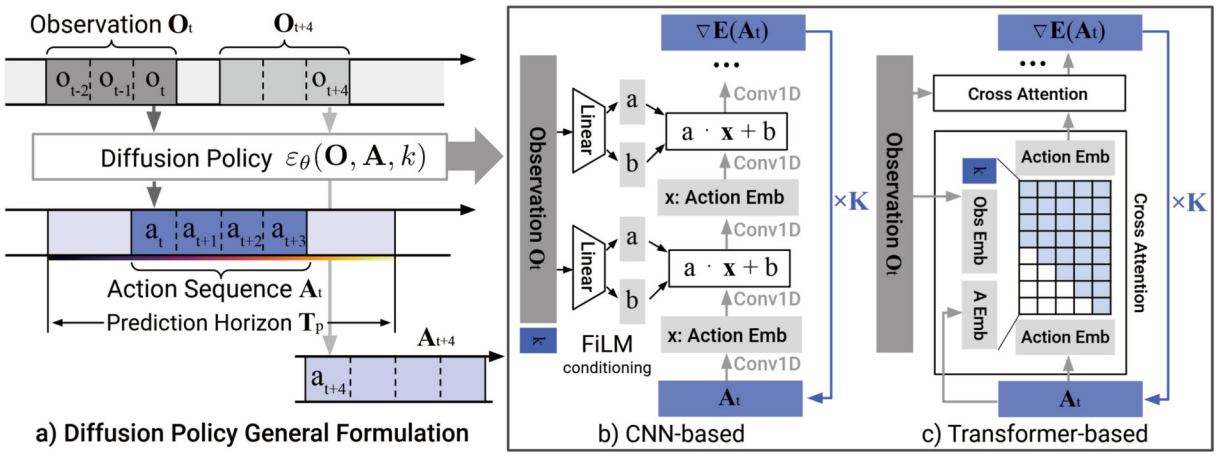



Fig. 7. (Color online) Diffusion policy general formulation and different model architectures (from Diffusion Policy[8]).

MLPs and cosine similarity calculations. Similarly, GaussianGrasper[89] employs GSNet to generate candidate grasping poses. It then selects the optimal pose by filtering out infeasible candidates through a surface normal guidance module based on force closure theory. In GraspNeRF[84], a CNN-based volumetric grasping detection module outputs grasping quality, rotation, and opening width through three independent decoder heads. Moreover, the YOSO grasping model[88] developed by Zhou *et al.* leverages the global information extraction capabilities of Transformers. YOSO utilizes a 3D instance segmentation network based on UNet to identify positive samples and a point Transformer-based encoder to generate grasping candidates. The final step involves grasping regression to produce the optimal grasping pose.

**(2) Action generation.** An action generation model outputs a single action or a sequence of actions directly from images and task descriptions. These models are typically trained from scratch on real robot trajectories collected through teleoperation by expert operators. Unlike cognitive planning models, which often comprise billions of parameters, action generation models are generally more compact. This reduction in size ensures faster inference times and higher control frequencies, enabling real-time responsiveness and precise execution of actions. In general, there are two types of action generation models based on different AI algorithms: autoregression-based models and diffusion-based models.

Autoregression-based models leverage the Transformer encoding-decoding process to handle long sequences, treating the mapping of language and vision observations to robot actions as a sequence modeling problem. One notable example is RT-1[93] developed by Google. RT-1 builds a Transformer-based model with 35 million parameters, trained on a large and diverse dataset of real-world robotic tasks. The model processes a history of six images by passing them through a pretrained EfficientNet-B3 model, while using feature-wise linear modulation (FiLM) layers to condition these visual tokens on language instructions. The visual-language tokens are then fed into the Transformer backbone, which includes a token learner to reduce the number of tokens processed, thereby speeding up inference and enabling real-time control. RT-1 also introduces an action tokenization method that maps continuous robot actions to discrete language tokens, facilitating efficient and scalable action generation. Another significant work is ALOHA[7] developed by Zhao *et al*. ALOHA employs a combination of convolutional neural networks (CNNs) and a Transformer encoder-decoder architecture with 80M parameters to learn from expert demonstrations of robot actions, as shown in Fig. 6. It uses four cameras to capture environmental changes and robot movements, which are processed by ResNet18 backbones[94]. By integrating visual features with current joint positions and task variables, the Transformer-based encoder and decoder generate a sequence of robot actions tailored to specific tasks. Additionally, ALOHA develops a low-cost, open-source teleoperation system designed for precise bimanual manipulation tasks, achieving high success rates in real-world fine manipulation tasks using affordable off-the-shelf hardware.

Diffusion-based models are another popular method for dexterous robot actions. As diffusion models have shown superior performance in modeling high dimensional data like images, it is motivated to represent a robot's action sequence generation as a conditional denoising diffusion process on robot action space. Diffusion policy[8] (Fig. 7) is the first work to apply denoising diffusion probabilistic models (DDPMs) to the robot action space. By combining techniques such as closed-loop action sequences, visual conditioning, and the time-series diffusion Transformer, it significantly outperforms other methods in 15 tasks, with an average success rate improvement of 46.9%. To support general tasks, UniPi[95] and actions from video dense correspondences (AVDC)[96] use a conditional diffusion model which takes the initial frame and text description as conditions to learn the distribution of future frames. By synthesizing videos that "hallucinate" a robot executing actions, these methods have shown their effectiveness in various robotic tasks and their ability to achieve cross-embodiment learning. A new state-of-the-art of diffusion-based models is RDT-1B[97], which leverages diffusion models and a scalable Transformer architecture, combined with a physically interpretable unified action space. It is pretrained on a large-scale multi-robot dataset and fine-tuned on a self-created multi-task bimanual dataset. It significantly outperforms existing methods in zero-shot generalization, instruction following, few-shot learning, and complex task handling, with an average success rate improvement of 56%.

### 3.1.3. Hierarchical framework

Based on the advancement of cognitive planning mod-

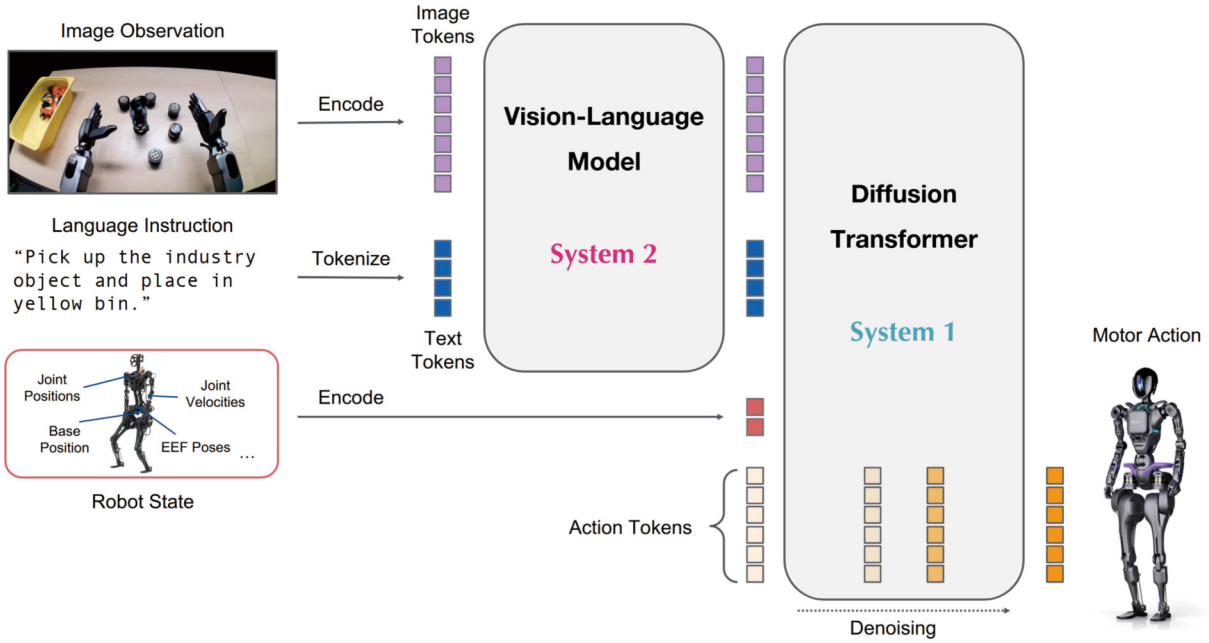



Fig. 8. (Color online) GR00T N1 model architecture overview (from GR00T N1[14]).

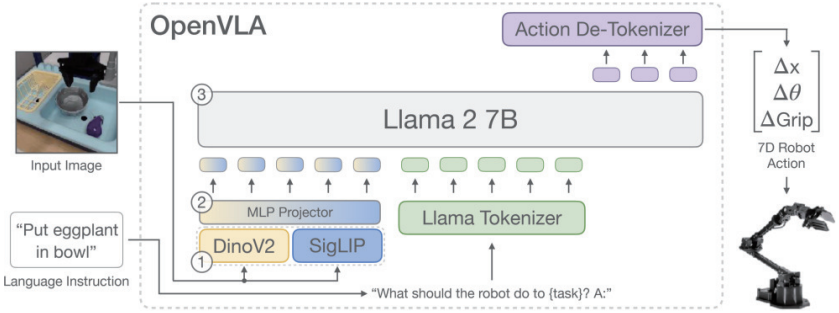



Fig. 9. (Color online) OpenVLA model architecture overview (from OpenVLA[17]).

els and action execution models, it is a natural idea to combine both models into a framework to gain the benefits of multimodal perception, long-horizon planning, and dexterous action. Therefore, many hierarchical frameworks have been proposed in recent months. SkillDiffuser[11] is a notable hierarchical framework from the academia. At the higher level, it uses the skill abstraction module adapted from GPT-2 to learn discrete, human-understandable skill representations from visual observations and language instructions. These learned skill embeddings are then used to condition the diffusion model to generate customized latent trajectories. $\pi_0$[12], developed by Physical Intelligence, is the first hierarchical model from the industry that incorporates a pre-trained VLM as its cognitive planning model and a diffusion-variant flow-matching model as its action execution model. It is pre-trained on a large-scale dataset containing data from 7 robot configurations, 68 tasks, and open-source data, and then fine-tuned for specific downstream tasks. Thus, it achieves a much higher success rate than baseline models in zero-shot tasks and outperforms other methods in fine-tuning and learning complex tasks, providing an effective solution for general robot control. GO-1[98], another hierarchical framework developed by AgiBot, comprises three models during inference: a pretrained VLM, a latent planner, and an action expert. The VLM first encodes multi-view image observations and language instructions into tokens, and then the latent planner performs embodiment-agnostic planning within the latent action space, and finally the action expert utilizes a diffusion objective to model the continuous distribution of low-level actions. With the proposed AgiBot World Colosseo, a large-scale manipulation platform, GO-1 shows a success rate of over 60% in complex tasks. GR00T N1[14], a VLA dual-system architecture developed by NVIDIA, also shows a hierarchical-like framework in Fig. 8. System 2 is based on a VLM to process visual perception and language instructions to understand the environment and task goals, and System 1 is based on a Diffusion Transformer Module to generate high-frequency actions based on the output of the VLM and the robot's state. In simulation and real-world experiments, GR00T N1 outperforms baseline models in various tasks, demonstrating strong generalization ability and data utilization efficiency. Another example is Helix[13], developed by Figure, which consists of 7B-VLM-based System 2 and 80M-Transformer-based System 1.

### 3.2. End-to-end model

An end-to-end model is possibly the ultimate step toward general-purpose robots and artificial general intelligence. These models are designed to seamlessly integrate perception, planning, and action into a single model, enabling robots to process multimodal contextual inputs—such as visual data and natural language—and execute general-purpose tasks in different domains with high levels of intelli-

gence in real time.

A primary approach to realizing end-to-end embodied models involves the use of vision-language-action models (VLAs), which process multimodal inputs from vision and language to generate robotic actions, demonstrating superior capabilities in long-horizon task planning and reasoning. RT-2[15], developed by Google, represents the first significant implementation of a VLA that integrates web-scale vision-language pretraining directly into robotic action generation. This integration enhances the model's generalization and semantic reasoning capabilities, particularly for robotic manipulation tasks. RT-2 leverages pretrained vision-language models (VLMs) as its backbone to leverage the generalizable multimodal and semantic knowledge from web data. It employs a ViT to project the images into the language token space, and a discretization technique proposed by RT-1[93] to map output tokens from VLM to robot action space. By jointly fine-tuning a VLM with robot trajectory data, RT-2 significantly improves its ability to generalize novel objects, environments, and task instructions. Additionally, RT-2 exhibits emergent capabilities, including multi-step reasoning and the interpretation of complex instructions, facilitated by chain-of-thought prompting. However, despite these advancements, RT-2's physical skillset remains limited to actions observed during training. Building upon RT-2, RT-X[16] introduces performance improvements by constructing a large robotic dataset known as Open X-Embodiment. This collaborative dataset aggregates data from 22 distinct robotic platforms across 21 institutions, encompassing 527 skills and 160 266 tasks. It develops basic VLAs, RT-1-X and RT-2-X, based on the architectures of RT-1[93] and RT-2[15], respectively and co-train them on the large embodiment dataset, thereby enhancing generalization across various robotic platforms and scenarios. Another significant advancement is made by OpenVLA[17], a state-of-the-art, open-source VLA model developed by Stanford University. OpenVLA is built upon the Prismatic-7B VLM[99], which comprises a 600M-parameter ViT visual encoder, a 2 MLP projector, and a 7B-parameter Llama2 backbone, as shown in Fig. 9. Furthermore, it introduces efficient fine-tuning methods, including low-rank adaptation and model quantization techniques, facilitating rapid adaptation to new robotic tasks and configurations.

## 3.3. Embodied AI hardware

Despite the various embodied AI models, these algorithms typically share common basic algorithms, providing opportunities for designing accelerators for these fundamental algorithms. Transformer is one of the most common building blocks composing the backbones of various embodied AI models such as end-to-end models and cognitive planning models. Its computation typically dominates the overall inference time, especially in end-to-end models with more than billions of parameters. Another basic algorithm is 3D reconstruction which is important for many action execution models, but it usually takes dozens of seconds to several minutes to reconstruct the target object and the entire scene, thus identified as the main bottleneck for fast action execution. Meanwhile, diffusion models are emerging as more attractive for action execution models, whereas their iterative denoising process may result in high computational latency compared with Transformer-based models of similar size. Overall, these core components—Transformers, 3D reconstruction, and diffusion models—represent the primary computational bottlenecks across diverse embodied AI algorithms. Therefore, designing dedicated hardware accelerators targeting these specific operations promises significant speedups and meets the energy consumption and resource limitations at the edge side.

### 3.3.1. Transformer accelerator

The challenge of Transformer acceleration derives from its self-attention mechanism, which has quadratic computational and memory complexity in terms of the number of tokens. This creates a significant bottleneck, especially for LLMs to process long task descriptions and VLMs to process high image resolutions. As a result, it is crucial to design a dedicated dataflow to minimize the computational overhead and external memory access. To address these challenges, many works have explored inherent algorithmic features such as data similarity and sparsity patterns and developed accelerators to leverage these properties effectively.

Recent advancements in digital accelerators for Transformers have focused on improving both the computational performance and energy efficiency of large Transformers. Wang *et al.* investigated sparsity patterns and data redundancy within the self-attention mechanisms of Transformers, particularly focusing on weakly related tokens[23]. By leveraging these patterns, they developed an approximate processing element equipped with a zero-speculation unit and an out-of-order computing scheduler. This innovative approach accelerates Transformer workloads and achieves substantial energy efficiency improvements by minimizing unnecessary computations and optimizing data flow. Tambe *et al.* took advantage of the entropy-based early exit algorithm to design a specialized datapath with mixed-precision computations and power management to reduce energy and latency, while maintaining high accuracy over conventional BERT inference[100]. Kim *et al.* proposed a novel model architecture termed C-Transformer[101]. This architecture integrates a DNN-Transformer and a spiking-Transformer within a big-little network framework and employs implicit weight generation. The C-Transformer significantly increases computational energy efficiency by eliminating the external memory access bottleneck. The accelerator features a homogeneous core for both Transformer types, with an output spike speculation unit and an implicit weight generation unit. These innovations enable the deployment of LLMs, such as GPT-2, on mobile devices, achieving state-of-the-art system performance while maintaining low energy consumption.

Compute-in-memory (CIM) accelerators offer another promising strategy for optimizing Transformer inference by integrating AI computation directly within memory. This integration significantly reduces external data transfer requirements and enhances energy efficiency, addressing one of the primary bottlenecks in Transformer models. For example, Tu *et al.* exploited pipeline opportunities within the self-attention mechanism to develop a reconfigurable streaming network tailored for attention layers[25]. It minimizes redundant external memory accesses by efficiently managing data flow. Additionally, they introduced a bitline-transpose CIM structure, which facilitates matrix transposing without the need for additional storage, and a sparse attention scheduler that

leverages block-wise sparsity patterns in attention layers. These innovations collectively contribute to reduced energy consumption and enhanced computational efficiency. In another study, Tu *et al.* focused on accelerating multimodal Transformers by addressing the hybrid sparsity patterns inherent in cross-modal attention mechanisms[102]. They identified hybrid attention-token-bit sparsity patterns and the associated challenges when implementing these patterns on CIM-based accelerators. To overcome these challenges, they developed a long reuse elimination scheduler that optimizes CIM utilization at the attention level. Furthermore, they introduced a runtime token pruner and a modal-adaptive CIM network to enhance performance during cross-model switching. They also presented an effective bitwidth-balanced CIM architecture that ensures balanced input bits across in-memory computations, thereby improving overall system performance. For the unified acceleration of both popular Transformers and CNNs, Guo *et al.* presented a novel hybrid analog-digital approach, integrating both analog and digital components within a 22 nm SRAM-based architecture[103]. This approach ensures high energy efficiency while maintaining inference accuracy through mechanisms like the 4 : 2 compressor-based adder tree and an analog-storage quantizer circuit. With its flexible accumulation lengths and reduced energy consumption, the lightning-like CIM macro offers promising results for accelerating Transformers, particularly in the context of high-precision, large-scale AI models.

#### 3.3.2. 3D reconstruction accelerator

3D reconstruction plays a critical role in embodied action execution models to create 3D geometric views from 2D inputs. However, traditional hardware platforms such as GPUs often struggle with inefficiency in 3D reconstruction tasks. This inefficiency stems from the two primary computational patterns in 3D reconstruction: intensive spatial computation and irregular memory access. To address these problems, many works have proposed approximate computing techniques to skip unnecessary computation and dataflow transformation techniques to rectify irregular data flow. According to the reconstruction methods, the corresponding accelerators can be divided into point cloud reconstruction accelerators, NeRF reconstruction accelerators, and 3D GS reconstruction accelerators.

Point cloud-based reconstruction calculates disparity maps from RGB(D) images and sensor calibration parameters to generate 3D point clouds. However, the irregular and sparse nature of point cloud data poses challenges for accelerator design, as it results in frequent external memory access and higher power consumption. Additionally, processing point cloud features involves complex calculations that require significant computational resources and energy. To solve this problem, Im *et al.* utilized the low-power ToF sensor to reconstruct dense depth information and further calculated bounding boxes in 3D views through point cloud feature processing, reducing power by more than 60%[104]. Specifically, they addressed the memory access bottleneck caused by conjugate gradient methods in the reconstruction process by employing a window-based technique that transforms large irregular sparse matrices into band matrices with diagonal regularity and using run-length encoding for compression. They also optimized point cloud feature processing by reusing features and skipping unnecessary calculations. To support efficient sparse convolution and neighbor search in point cloud networks, Sun *et al.* presented a reconfigurable sparse convolution core that supports different types of sparse convolutions through dual dataflow reconfiguration[105], along with a neighbor search circuit based on priority codes to transform the search problem into finding the rightmost non-zero index. Moreover, Jung *et al.* further enhanced neighbor search efficiency by embedding a pseudo-random number generator (PRNG) circuit to predict search problems and adjust sampling rates based on frame familiarity[106].

NeRF excels at creating high-quality, hyper-realistic 3D models but requires significant computational resources. This challenge lies in the intensive calculations performed by MLPs, which consume nearly 97% of the processing time and demand substantial memory access. To address this problem, Han *et al.* proposed MetaVRain, a real-time and energy-efficient NeRF architecture[107]. MetaVRain employs three key stages to optimize computational efficiency: in the spatial attention stage, only ray-wise samples are collected within the voxel attention map, skipping calculations in other regions. In the temporal familiarity stage, partial MLP is used to calculate the RGB-D pixel color difference between consecutive frames, enabling the system to skip calculations for frames with high familiarity, thus reducing computation by over 95%. The final top-down attention stage further minimizes computation by skipping network layers and samples, relying on density calculations from previous layers. In addition, the positional encoding unit is approximated by a periodic polynomial to calculate the sinusoidal function in NeRF. Although recent NeRF models have been optimized through hash embedding techniques for faster modeling speed, they still require significant modeling time (over 10 min on GPUs). To reduce this latency, Ryu *et al.* introduced NeuGPU, an efficient architecture that achieves a modeling time of just 34 s, substantially outperforming edge GPUs[108]. NeuGPU achieves this performance by fully storing hash tables on-chip using segmented hashing with spatial pruning. It also uses attention-based hybrid interpolation to skip vertices with minimal contributions and employs similarity sparsity to skip similar activations with zero input or negligible differences. Park *et al.* further advanced NeRF-based reconstruction by leveraging the small memory footprint of NeRF to optimize SLAM-based reconstruction frameworks. They accelerated this NeRF-SLAM reconstruction system with an out-of-order SMoE router and a single-skip (SS) and dual-skip (DS) heterogeneous core to exploit coarse-grained sparsity[109].

Gaussian splatting (GS) offers a promising alternative to NeRF by utilizing rasterization techniques. Despite this, real-time processing remains challenging, with systems like the Jetson Xavier GPU achieving only 6.4 fps. Research by Lee *et al.*[110] highlights that the main bottlenecks lie in the sorting and rasterization processes, which slow down the computational pipeline. To address these issues, they proposed the GSCore hardware acceleration unit, which achieved a processing speed of 91.2 fps[110]. Specifically, the GSCore uses a Gaussian shape-aware intersection test to identify suitable bounding boxes, thereby reducing false positives. It also uses a two-stage hierarchical sorting process that approximates sorting across all Gaussians and refines the sorting during rasteriza-

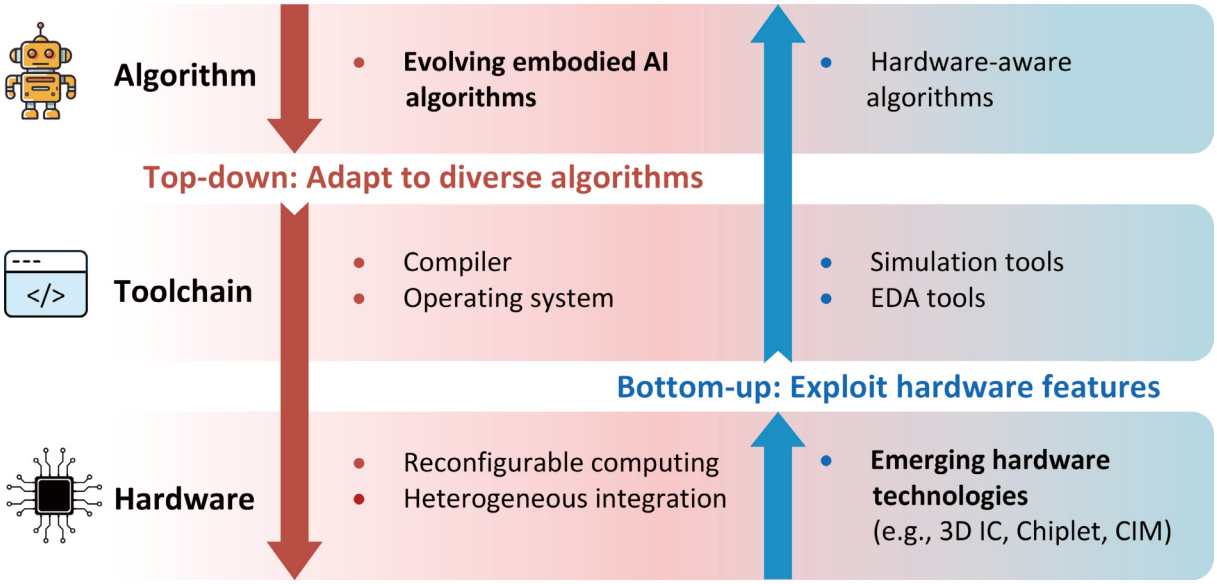



Fig. 10. (Color online) The co-evolution of algorithms and hardware raises two challenges: (1) From the top down, adapt computing platforms to evolving embodied AI algorithms. (2) From the bottom up, transform the potential of emerging hardware innovations (e.g., 3D IC, Chiplet, CIM) into end-to-end inference improvements.

tion, skipping unnecessary computations. In addition, the combination of 3D GS and SLAM algorithms is becoming more attractive to improve SLAM-based reconstruction frameworks with high fidelity and explicit representation provided by 3D GS, though volume rendering remains a bottleneck for tracking throughput. To overcome this, Wu *et al.* developed a sparse-tile-sampling method that trains pose parameters using informative regions while discarding others[111]. Based on this approach, they designed the sparsity-aware processor GauSPU, which achieved a 63.9× improvement in energy efficiency compared to the RTX 3090 GPU.

#### 3.3.3. Diffusion model accelerator

The iterative nature of diffusion models introduces significant challenges for acceleration as the computation complexity and memory access will increase linearly with the iteration counts. To address the problem, many works have focused on optimizing the dataflow across adjacent iterations. Specifically, to improve hardware efficiency, several hardware accelerators leverage differential computation to accelerate diffusion models based on the similarity of input data across adjacent iterations. These accelerators include both digital-based and CIM-based approaches.

Differential computation is effective for accelerating linear operations, but for nonlinear operators such as activation functions, it requires repeatedly loading large bit-width raw inputs into the chip, resulting in heavy memory access. To address this issue, Kong *et al.* proposed the Cambricon-D accelerator, which optimizes both memory access and outlier calculations, achieving a reduction of more than 66% in memory access[112]. The accelerator employs several techniques to mitigate memory bottlenecks: first, a sign-mask dataflow is utilized to calculate nonlinear activation functions by loading only the 1-bit sign of inputs, thus avoiding the need for large bit-width raw inputs. Second, to handle outliers, an outlier-aware processing element (PE) array quantizes input values into a large set of inliers and a small set of outliers. These inliers and outliers are processed using integer and floating-point multipliers, respectively, optimizing computational efficiency.

Given the high throughput and energy efficiency of CIM, Guo *et al.* proposed a CIM-based diffusion model accelerator, which can achieve a system energy efficiency of 74.34 TFLOPS/W[113]. Specifically, this design divides input variations into dense integer and sparse floating-point computations to optimize performance. For integer operations, a radix-8 Booth CIM macro is employed to perform 4-bit octal computation, which reduces power consumption by over 30% compared to traditional bit-serial CIM macros. For floating-point operations, the accelerator uses a reconfigurable 4-operand exponent CIM (4Op-ECIM) macro. This macro performs two addition operations on four bitcells and processes the mantissa data through a dedicated mantissa processing engine. Additionally, the 4Op-ECIM can be configured as a binary content-addressable memory (CAM) to identify and skip redundant calculations for sparse floating-point values, further improving computational efficiency.

## 4. Conclusion and future research

In this survey, we provide a comprehensive overview of robotic computing systems from both robotic algorithms and computing hardware, highlighting the algorithm-hardware co-design perspective to achieve balanced performance. The evolution of robotics and embodied AI is summarized into a roadmap from traditional robotics to hierarchical and end-to-end embodied AI models. Despite the significant advancements, achieving an appropriate balance among system metrics such as accuracy, latency, and power consumption remains challenging. To address this, we emphasize the algorithm-hardware co-design methodology, which analyzes common algorithm computational behaviors on hardware and optimizes them through algorithm optimization and hardware innovation. We then reviewed recent works on robotic and embodied AI algorithms and computing hardware to demonstrate this algorithm-hardware co-design methodology.

As the era of embodied AI has just arrived, there are new challenges and many research opportunities for further explo-

ration in the direction of realizing smarter and faster robotic computing systems. On the algorithm side, embodied AI algorithms are rapidly evolving to handle more complex robotic tasks and extend to more general scenarios. On the hardware side, with the slowing down of Moore's Law, emerging hardware technology such as 3D IC, Chiplet, and CIM are being developed to deliver higher performance and energy efficiency. As illustrated in Fig. 10, these trends result in two new challenges in the algorithm-hardware co-design of embodied AI computing systems: (1) adapting computing platforms to the rapid evolution of embodied AI algorithms, and (2) transforming the potential of emerging hardware innovations into end-to-end inference improvements.

From the top down, the rapidly evolving embodied AI algorithms demand the adaptation of the software toolchain and hardware architecture. Specifically, the software toolchain is required to support the fast-changing algorithm architectures and dynamic configurations of sensors and actuators. These problems can be addressed through compilers and operating systems:

• Compilers can be developed into modular architectures to separate different optimization techniques and code transformations into independent components, enabling simple updating of new model structures and operators without excessive rework.

• Operating systems can provide software interfaces for sensors and actuators and manage their data synchronization autonomously, simplifying the process of transferring algorithms between different configurations.

Meanwhile, the hardware architecture must efficiently support the evolution of diverse algorithms. In this context, dynamic performance requirements and diverse specialized applications pose obstacles to hardware flexibility. Reconfigurable computing and heterogeneous integration are two promising techniques to overcome them:

• Reconfigurable hardware can adjust the hardware settings like dataflow and computing precisions at runtime to meet the dynamic performance demands such as low latency for control algorithms and high energy efficiency for planning models.

• Heterogeneous integration can incorporate high-performance hardware tailored to different algorithms seamlessly within a short period of time, enabling agile deployment for various applications requiring different algorithms.

From the bottom up, transforming the underlying potential of emerging hardware technologies into end-to-end inference improvements requires innovations in the hardware toolchain and algorithms. Here, the performance evaluation of the emerging technologies and the expanded design space presents challenges for the hardware toolchain. Simulation tools and electronic design automation (EDA) tools are two key areas that need further exploration:

• Simulation tools should provide precise models for the physical characteristics of these new advancements and simulate their performance within a reasonable period, enabling accurate and quick evaluation of the deployment efficiency for programmers.

• EDA tools should explore the enlarged design space based on these emerging technologies to exploit their hardware benefits, allowing researchers to design next-generation embodied AI platforms effectively.

Furthermore, from the perspective of embodied AI algorithms, these cutting-edge hardware technologies introduce unique computation features for algorithm processing. For example, 3D IC offers ultra-high bandwidth through massive vertical vias, but cross-bank communication is relatively slow; Chiplet technology provides flexible heterogeneous integration and enables larger chip size beyond the reticle limit; CIM breaks the Von Neumann bottleneck by integrating compute into memory. These features can be exploited directly in algorithm development to improve end-to-end inference performance. For example, motivated by the bank-wise ultra-high bandwidth of 3D IC, new embodied AI training algorithms may be explored to partition the gradient computing and parameter updating into bank granularities with minimum cross-bank communication, enabling high-performance training on 3D IC. Embracing the evolution of algorithms and hardware technologies, the cross-stack co-design methodology will be significant for the development of next-generation robotic computing systems.

## Acknowledgments

This work was supported in part by NSFC under Grant 62422407; in part by RGC under Grant 26204424; and in part by ACCESS–AI Chip Center for Emerging Smart Systems, sponsored by the InnoHK initiative of the Innovation and Technology Commission of the Hong Kong Special Administrative Region Government.

## References

[1] Attanasio A, Scaglioni B, Momi E D, et al. Autonomy in surgical robotics. Annu Rev Control Rob Auton Syst, 2021, 4, 651
[2] Guizzo E. Universal robots introduces its strongest robotic arm yet. 2019
[3] Wan S, Goudos S. Faster r-cnn for multi-class fruit detection using a robotic vision system. Comput Networks, 2020, 168, 107036
[4] LaValle S. Rapidly-exploring random trees : a new tool for path planning. Annu Res Rep, 1998
[5] Hart P E, Nilsson N J, Raphael B. A formal basis for the heuristic determination of minimum cost paths. IEEE Trans Syst Sci Cybern, 1968, 4, 100
[6] Fang HS, Wang C, Fang H, et al. Anygrasp: robust and efficient grasp perception in spatial and temporal domains. IEEE Trans Robot, 2023, 39, 39293
[7] Zhao T Z, Kumar V, Levine S, et al. Learning fine-grained bimanual manipulation with low-cost hardware. arXiv, 2023
[8] Chi C, Xu Z, Feng S, et al. Diffusion policy: visuomotor policy learning via action diffusion. arXiv, 2023
[9] Vemprala S, Bonatti R, Bucker A, et al. ChatGPT for robotics: design principles and model abilities. arXiv, 2023
[10] Driess D, Xia F, Sajjadi M S M, et al. PaLM-e: an embodied multimodal language model. arXiv, 2023
[11] Liang Z, Mu Y, Ma H, et al. SkillDiffuser: interpretable hierarchical planning via skill abstractions in diffusion-based task execution. arXiv, 2024
[12] Black K, Brown N, Driess D, et al. $\pi_0$: a vision-language-action flow model for general robot control. arXiv, 2024
[13] Figure. Helix: a vision-language-action model for generalist humanoid control. Figure, 2025
[14] NVIDIA, Bjorck J, Castañeda F, et al. GR00T n1: an open foundation model for generalist humanoid robots. arXiv, 2025
[15] Brohan A, Brown N, Carbajal J, et al. RT-2: vision-language-ac-

tion models transfer web knowledge to robotic control. arXiv, 2023
[16] Collaboration O X E, O'Neill A, Rehman A, et al. Open x-embodiment: robotic learning datasets and rt-x models. arXiv, 2023
[17] Kim M J, Pertsch K, Karamcheti S, et al. OpenVLA: an open-source vision-language-action model. arXiv, 2024
[18] Zhao Q, Lu Y, Kim M J, et al. CoT-vla: visual chain-of-thought reasoning for vision-language-action models. arXiv, 2025
[19] Karaman S, Frazzoli E. Sampling-based algorithms for optimal motion planning. arXiv, 2011
[20] Wan Z, Yu B, Li T Y, et al. A survey of fpga-based robotic computing. IEEE Circuits Syst Mag, 2021, 21, 48
[21] Bakhshalipour M, Gibbons P B. Agents of autonomy: a systematic study of robotics on modern hardware. Proc, ACM Meas, Anal, Comput, Syst, 2023, 4(1), 651
[22] Suleiman A, Zhang Z, Carlone L, et al. Navion: a 2-mw fully integrated real-time visual-inertial odometry accelerator for autonomous navigation of nano drones. IEEE J Solid-State Circuits, 2019, 54, 1106
[23] Wang Y, Qin Y, Deng D, et al. A 28nm 27.5tops/w approximate-computing-based transformer processor with asymptotic sparsity speculating and out-of-order computing. 2022 IEEE International Solid-State Circuits Conference (ISSCC), 2022, 1
[24] Li Z, Dong Q, Saligane M, et al. 3.7 a 1920×1080 30fps 2.3tops/w stereo-depth processor for robust autonomous navigation. 2017 IEEE International Solid-State Circuits Conference (ISSCC), 2017, 62
[25] Tu F, Wu Z, Wang Y, et al. A 28nm 15.59μj/token full-digital bitline-transpose cim-based sparse transformer accelerator with pipeline/parallel reconfigurable modes. 2022 IEEE International Solid-State Circuits Conference (ISSCC), 2022, 466
[26] Tesla. Tesla's optimus. Tesla
[27] Agility Robotics. Digit supports several manufacturing specific workflows. Agility Robotics
[28] Hmidani O, Ismaili Alaoui E M. A comprehensive survey of the r-cnn family for object detection. 2022 5th International Conference on Advanced Communication Technologies and Networking (CommNet), 2022, 1
[29] Nazir A, Wani Mohd A. You only look once-object detection models: a review. 2023 10th International Conference on Computing for Sustainable Global Development (INDIACom), 2023, 1088
[30] Lefebvre M, Moreau L, Dekimpe R, et al. A 0.2-to-3.6 tops/w programmable convolutional imager soc with in-sensor current-domain ternary-weighted mac operations for feature extraction and region-of-interest detection. 2021 IEEE international solid-state circuits conference (ISSCC), 2021, 118
[31] Gong Y, Zhang T, Guo H, et al. 22.7 dl-vopu: an energy-efficient domain-specific deep-learning-based visual object processing unit supporting multi-scale semantic feature extraction for mobile object detection/tracking applications. 2023 IEEE international solid-state circuits conference (ISSCC), 2023, 1
[32] Ronneberger O, Fischer P, Brox T. U-net: convolutional networks for biomedical image segmentation. Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, 2015, 234
[33] Chen L C, Papandreou G, Kokkinos I, et al. Deeplab: semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. IEEE Trans Pattern Anal Mach Intell, 2017, 40, 834
[34] Vohra J, Gupta A, Alioto M. 6.3 imager with in-sensor event detection and morphological transformations with 2.9 pj/pixel× frame object segmentation fom for always-on surveillance in 40nm. 2024 IEEE international solid-state circuits conference (ISSCC), 2024, 104
[35] Guo A, Si X, Chen X, et al. A 28nm 64-kb 31.6-tflops/w digital-domain floating-point-computing-unit and double-bit 6t-sram computing-in-memory macro for floating-point cnns. 2023 IEEE international solid-state circuits conference (ISSCC), 2023, 128
[36] Yoon S, Park S K, Kang S, et al. Fast correlation-based stereo matching with the reduction of systematic errors. Pattern Recognit Lett, 2005, 26, 2221
[37] Hirschmuller H. Accurate and efficient stereo processing by semi-global matching and mutual information. 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05), 2005, 807
[38] Sekhar V C, Bora S, Das M, et al. Design and implementation of blind assistance system using real time stereo vision algorithms. 2016 29th International Conference on VLSI Design and 2016 15th International Conference on Embedded Systems (VLSID), 2016, 421
[39] Li M, Mourikis A I. High-precision, consistent ekf-based visual-inertial odometry. Int, J Rob, Res, 2013, 32, 690
[40] Liu Q, Hao Y, Liu W, et al. An energy efficient and runtime reconfigurable accelerator for robotic localization. IEEE Trans Comput, 2023, 72, 1943
[41] Gan Y, Bo Y, Tian B, et al. Eudoxus: characterizing and accelerating localization in autonomous machines industry track paper. 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA), 2021, 827
[42] Liu W, Yu B, Gan Y, et al. Archytas: a framework for synthesizing and dynamically optimizing accelerators for robotic localization. MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture, 2021, 479
[43] Paden B, Čáp M, Yong S Z, et al. A survey of motion planning and control techniques for self-driving urban vehicles. IEEE Trans Intell Veh, 2016, 1, 33
[44] Urmson C, Anhalt J, Bagnell D, et al. Tartan racing: a multi-modal approach to the darpa urban challenge. DARPA Grand Challenge Tech Report, 2007
[45] Liu S, Zheng K, Zhao L, et al. A driving intention prediction method based on hidden markov model for autonomous driving. arXiv, 2019
[46] Talpes E, Sarma D D, Venkataramanan G, et al. Compute solution for tesla's full self-driving computer. IEEE Micro, 2020, 40, 25
[47] González D, Pérez J, Milanés V, et al. A review of motion planning techniques for automated vehicles. IEEE Trans Intell Transp Syst, 2016, 17, 1135
[48] Dijkstra E W. A note on two problems in connexion with graphs. Numer, Math, 1959, 1, 269
[49] Koppen M. The curse of dimensionality. Proceedings of the 5th Online World Conference on Soft Computing in Industrial Applications, 2000, 4
[50] Kavraki L E, Svestka P, Latombe J C, et al. Probabilistic roadmaps for path planning in high-dimensional configuration spaces. IEEE Trans Robot Autom, 1996, 12, 566
[51] Bakhshalipour M, Ehsani S B, Qadri M, et al. RACOD: algorithm/hardware co-design for mobile robot path planning. Proceedings of the 49th Annual International Symposium on Computer Architecture, 2022, 597
[52] Lian S, Han Y, Chen X, et al. Dadu-p: a scalable accelerator for robot motion planning in a dynamic environment. 2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC), 2018, 1
[53] Chung C, Yang C H. A 1.5-μj/task path-planning processor for 2-d/3-d autonomous navigation of microrobots. IEEE J Solid-State Circuits, 2021, 56, 112
[54] Gasparetto A, Boscariol P, Lanzutti A, et al. Path planning and trajectory planning algorithms: a general overview. Motion and Operation Planning of Robotic Systems: Background and Practical Approaches, 2015, 3

[55] Betts J T. Survey of numerical methods for trajectory optimization. J Guid Control Dyn, 1998, 21, 193
[56] Boyd S, Parikh N, Chu E, et al. Distributed optimization and statistical learning via the alternating direction method of multipliers. Found Trends® Mach Learn, 2011, 3, 1
[57] Zhao Z, Cheng S, Ding Y, et al. A survey of optimization-based task and motion planning: from classical to learning approaches. IEEE/ASME Trans Mechatronics, 2024, 1
[58] Chrétien B, Escande A, Kheddar A. GPU robot motion planning using semi-infinite nonlinear programming. IEEE Trans Parallel Distrib Syst, 2016, 27, 2926
[59] Hao Y, Gan Y, Yu B, et al. BLITZCRANK: factor graph accelerator for motion planning. 2023 60th ACM/IEEE Design Automation Conference (DAC), 2023, 1
[60] Hao Y, Gan Y, Yu B, et al. ORIANNA: an accelerator generation framework for optimization-based robotic applications. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, 2024, 2, 813
[61] Aristidou A, Lasenby J, Chrysanthou Y, et al. Inverse kinematics techniques in computer graphics: a survey. Comput Graphics Forum, 2018, 37, 35
[62] Tkachenko O, Song K T. Hardware accelerated inverse kinematics for low power surgical manipulators. 2020 International Conference on Advanced Robotics and Intelligent Systems (ARIS), 2020, 1
[63] Lian S, Han Y, Wang Y, et al. Dadu: accelerating inverse kinematics for high-dof robots. Proceedings of the 54th Annual Design Automation Conference 2017, 2017, 1
[64] Plancher B, Neuman S M, Ghosal R, et al. GRiD: gpu-accelerated rigid body dynamics with analytical gradients. 2022 International Conference on Robotics and Automation (ICRA), 2022, 6253
[65] Neuman S M, Koolen T, Drean J, et al. Benchmarking and workload analysis of robot dynamics algorithms. 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2019, 5235
[66] Neuman S M, Plancher B, Bourgeat T, et al. Robomorphic computing: a design methodology for domain-specific accelerators parameterized by robot morphology. Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, 2021, 674
[67] Neuman S M, Ghosal R, Bourgeat T, et al. RoboShape: using topology patterns to scalably and flexibly deploy accelerators across robots. Proceedings of the 50th Annual International Symposium on Computer Architecture, 2023, 1
[68] Yang Y, Chen X, Han Y. Dadu-rbd: robot rigid body dynamics accelerator with multifunctional pipelines. Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture, 2023, 297
[69] Liu W, Hua M, Deng Z, et al. A systematic survey of control techniques and applications in connected and automated vehicles. IEEE Internet Things J, 2023, 10, 21892
[70] Moro F L, Sentis L. Whole-body control of humanoid robots. Humanoid Robotics: A Reference, 2019, 1161
[71] Sentis L, Khatib O. Synthesis of whole-body behaviors through hierarchical control of behavioral primitives. Int J Humanoid Rob, 2005, 02, 505
[72] Dietrich A, Ott C, Albu-Schäffer A. An overview of null space projections for redundant, torque-controlled robots. Int J Rob Res, 2015, 34, 1385 (in Chinese)
[73] Dario Bellicoso C, Jenelten F, Fankhauser P, et al. Dynamic locomotion and whole-body control for quadrupedal robots. 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2017, 3359
[74] Sleiman J P, Farshidian F, Minniti M V, et al. A unified mpc framework for whole-body dynamic locomotion and manipulation. IEEE Robot Autom Lett, 2021, 6, 4688
[75] Ngo H Q T, Duc Nguven H, Truong Q V. A design of pid controller using fpga-realization for motion control systems. 2020 International Conference on Advanced Computing and Applications (ACOMP), 2020, 150
[76] Li Y, Li S E, Jia X, et al. FPGA accelerated model predictive control for autonomous driving. J Intell Connected Veh, 2022, 5, 63
[77] Lin I T, Fu Z S, Chen W C, et al. 2.5 a 28nm 142mw motion-control soc for autonomous mobile robots. 2023 IEEE International Solid-State Circuits Conference (ISSCC), 2023, 1
[78] Huang J, Yong S, Ma X, et al. An embodied generalist agent in 3d world. arXiv, 2023
[79] Kim M J, Finn C, Liang P. Fine-tuning vision-language-action models: optimizing speed and success. arXiv, 2025
[80] Huang W, Xia F, Xiao T, et al. Inner monologue: embodied reasoning through planning with language models. arXiv, 2022
[81] Chowdhery A, Narang S, Devlin J, et al. PaLM: scaling language modeling with pathways. arXiv, 2022
[82] Dehghani M, Djolonga J, Mustafa B, et al. Scaling vision transformers to 22 billion parameters. arXiv, 2023
[83] Samavati T, Soryani M. Deep learning-based 3d reconstruction: a survey. Artif Intell Rev, 2023, 56, 9175
[84] Dai Q, Zhu Y, Geng Y, et al. Graspnerf: multiview-based 6-dof grasp detection for transparent and specular objects using generalizable nerf. 2023 IEEE international conference on robotics and automation (ICRA), 2023, 1757
[85] Shi J, A Y, Jin Y, et al. ASGrasp: generalizable transparent object reconstruction and grasping from rgb-d active stereo camera. arXiv, 2024
[86] Shi J, Yong A, Jin Y, et al. Asgrasp: generalizable transparent object reconstruction and 6-dof grasp detection from rgb-d active stereo camera. 2024 IEEE international conference on robotics and automation (ICRA), 2024, 5441
[87] Qi C R, Su H, Mo K, et al. PointNet: deep learning on point sets for 3d classification and segmentation. arXiv, 2017
[88] Zhou L, Wang H, Zhang Z, et al. You only scan once: a dynamic scene reconstruction pipeline for 6-dof robotic grasping of novel objects. 2024 IEEE international conference on robotics and automation (ICRA), 2024, 13891
[89] Zheng Y, Chen X, Zheng Y, et al. GaussianGrasper: 3d language gaussian splatting for open-vocabulary robotic grasping. arXiv prepr arXiv: 2403, 2024, 9(9), 7827
[90] Ze Y, Yan G, Wu Y H, et al. Gnfactor: multi-task real robot learning with generalizable neural feature fields. Conference on robot learning, 2023, 284
[91] Huang S, Chen L, Zhou P, et al. EnerVerse: envisioning embodied future space for robotics manipulation. arXiv, 2025
[92] Wang C, Fang H S, Gou M, et al. Graspness discovery in clutters for fast and accurate grasp detection. Proceedings of the IEEE/CVF international conference on computer vision, 2021, 15964
[93] Brohan A, Brown N, Carbajal J, et al. RT-1: robotics transformer for real-world control at scale. arXiv, 2023
[94] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition. 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, 770
[95] Du Y, Yang S, Dai B, et al. Learning universal policies via text-guided video generation. Adv Neural Inf Process Syst, 2023, 36, 9156
[96] Ko P C, Mao J, Du Y, et al. Learning to act from actionless videos through dense correspondences. arXiv, 2023
[97] Liu S, Wu L, Li B, et al. RDT-1b: a diffusion foundation model for bimanual manipulation. arXiv, 2024
[98] Bu Q, Cai J, Chen L, et al. AgiBot world colosseo: a large-scale

manipulation platform for scalable and intelligent embodied systems. arXiv, 2025

[99] Karamcheti S, Nair S, Balakrishna A, et al. Prismatic vlms: investigating the design space of visually-conditioned language models. arXiv, 2024

[100] Tambe T, Zhang J, Hooper C, et al. 22.9 a 12nm 18.1tflops/w sparse transformer processor with entropy-based early exit, mixed-precision predication and fine-grained power management. 2023 IEEE International Solid-State Circuits Conference (ISSCC), 2023, 342

[101] Kim S, Kim S, Jo W, et al. 20.5 c-transformer: a 2.6-18.1μj/token homogeneous dnn-transformer/spiking-transformer processor with big-little network and implicit weight generation for large language models. 2024 IEEE International Solid-State Circuits Conference (ISSCC), 2024, 368

[102] Tu F, Wu Z, Wang Y, et al. 16.1 muitcim: a 28nm 2.24 μJ/token attention-token-bit hybrid sparse digital cim-based accelerator for multimodal transformers. 2023 IEEE International Solid-State Circuits Conference (ISSCC), 2023, 248

[103] Guo A, Chen X, Dong F, et al. 34.3 a 22nm 64kb lightning-like hybrid computing-in-memory macro with a compressed adder tree and analog-storage quantizers for transformer and cnns. 2024 IEEE International Solid-State Circuits Conference (ISSCC), 2024, 570

[104] Im D, Park G, Li Z, et al. DSPU: a 281.6 mw real-time depth signal processing unit for deep learning-based dense rgb-d data acquisition with depth fusion and 3d bounding box extraction in mobile platforms. 2022 IEEE international solid-state circuits conference (ISSCC), 2022, 510

[105] Sun W, Feng X, Tang C, et al. A 28nm 2d/3d unified sparse convolution accelerator with block-wise neighbor searcher for large-scaled voxel-based point cloud network. 2023 IEEE international solid-state circuits conference (ISSCC), 2023, 328

[106] Jung J, Kim S, Seo B, et al. 20.6 lspu: a fully integrated real-time lidar-slam soc with point-neural-network segmentation and multi-level knn acceleration. 2024 IEEE International Solid-State Circuits Conference (ISSCC), 2024, 370

[107] Han D, Ryu J, Kim S, et al. 2.7 metavrain: a 133mw real-time hyper-realistic 3d-nerf processor with 1d-2d hybrid-neural engines for metaverse on mobile devices. 2023 IEEE international solid-state circuits conference (ISSCC), 2023, 50

[108] Ryu J, Kwon H, Park W, et al. 20.7 neugpu: a 18.5 mj/iter neural-graphics processing unit for instant-modeling and real-time rendering with segmented-hashing architecture. 2024 IEEE international solid-state circuits conference (ISSCC), 2024, 372

[109] Park G, Song S, Sang H, et al. 20.8 space-mate: a 303.5mw real-time sparse mixture-of-experts-based nerf-slam processor for mobile spatial computing. 2024 IEEE International Solid-State Circuits Conference (ISSCC), 2024, 374

[110] Lee J, Lee S, Lee J, et al. GSCore: efficient radiance field rendering via architectural support for 3d gaussian splatting. Proceedings of the 29th ACM international conference on architectural support for programming languages and operating systems, 2024, 3, 497

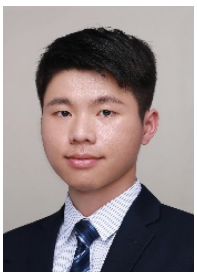
[111] Wu L, Zhu H, He S, et al. GauSPU: 3d gaussian splatting processor for real-time slam systems. 2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO), 2024, 1562

[112] Kong W, Hao Y, Guo Q, et al. Cambricon-d: full-network differential acceleration for diffusion models. 2024 ACM/IEEE 51st annual international symposium on computer architecture (ISCA), 2024, 903

[113] Guo R, Wang L, Chen X, et al. 20.2 a 28nm 74.34 tflops/w bf16 heterogenous cim-based accelerator exploiting denoising-similarity for diffusion models. 2024 IEEE international solid-state circuits conference (ISSCC), 2024, 362



**Longke Yan** received the B.S. degree from School of Integrated Circuit Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China, in 2024. He is currently pursuing the Ph.D. degree with Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong, China. His research interests include computer architecture, VLSI design, embodied AI, and algorithm-hardware co-design.



**Fengbin Tu** received the B.S. degree from the School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing, China, in 2013, and received the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, Beijing, China, in 2019. Dr. Tu is currently an Assistant Professor at the Department of Electronic and Computer Engineering and the Associate Director of the Institute of Integrated Circuits and Systems, The Hong Kong University of Science and Technology, Hong Kong, China. He was a Postdoctoral Fellow at the AI Chip Center for Emerging Smart Systems (ACCESS), Hong Kong, China, from 2022 to 2023, and a Postdoctoral Scholar at the Scalable Energy-efficient Architecture Lab (SEAL), the Department of Electrical and Computer Engineering, University of California, Santa Barbara, CA, USA, from 2019 to 2022. His research interests include AI chip, computing-in-memory, computer architecture, and reconfigurable computing. His Ph.D. thesis was recognized by the Tsinghua Excellent Dissertation Award. His AI chips ReDCIM and Thinker won the 2023 Top-10 Research Advances in China Semiconductors and 2017 ISLPED Design Contest Award. His research has been published at top conferences and journals on integrated circuits and computer architecture, including ISSCC, JSSC, DAC, ISCA, and MICRO.